# LakeTown Butchery — End-to-End Guide

**Version:** 1.0 (generated from project documentation)
**Purpose:** Run the stack locally (or in production), complete first-time setup, and operate a full business day from open to close.

---

## Table of contents


1. What you will achieve
2. Prerequisites
3. Part A — Run the application locally (100% end-to-end)
4. Part B — Verify the stack
5. Part C — First login and security
6. Part D — Admin setup (create everything in the right order)
7. Part E — Cashier day (operate one branch)
8. Part F — Admin oversight (reports and corrections)
9. Part G — Production deployment (summary)
10. Appendix


---

## 1. What you will achieve

By following this guide in order you will:

- Start **PostgreSQL**, **NestJS API**, and **Next.js** together (Docker).
- Open the **dashboard**, log in, and reach **Swagger API docs**.
- Complete **admin setup**: branches, users, products, recipes, suppliers, customers, partners.
- Run a **full cashier day**: stock session → stock → production → sales → payments → close.
- Know how to **deploy** using production compose and environment files (high level).

---

## 2. Prerequisites

| Requirement | Notes |
|---|---|
| **Docker** and **Docker Compose** | Required for the scripted path below. |
| **Git** | To clone the repository. |

| Requirement | Notes |
|---|---|
| **Ports free** | Defaults: **4515** (API), **3015** (frontend), **5434** (Postgres on host). Change via `.env` if needed. |
| **Browser** | Modern Chrome/Firefox/Edge for the dashboard. |

**Optional:** `curl` to hit the health endpoint from a terminal.

---

## 3. Part A — Run the application locally (100% end-to-end)

### Step A1 — Get the code

```
git clone <your-repository-url> laketown
cd laketown
```

### Step A2 — Environment (optional but recommended)

The dev stack reads from **`.env`** in the **repository root** (same folder as `docker-compose.yml`). If the file is missing, defaults apply (see Appendix).

Minimum you may set for local dev:

```
# Example .env at repo root (optional)
DATABASE_USER=laketown
DATABASE_PASSWORD=laketown_pass
DATABASE_NAME=laketown_butchery
JWT_SECRET=default_secret_key_change_in_production
JWT_EXPIRES_IN=7d
FRONTEND_URL=http://localhost:3015
NEXT_PUBLIC_API_URL=http://localhost:4515/api
```

Ensure **`NEXT_PUBLIC_API_URL`** matches **`BACKEND_HOST_PORT`**: if the API is on port **4515**, use `http://localhost:4515/api`.

### Step A3 — Start everything

#### Option 1 — Helper script (recommended)

```
chmod +x .dev.sh
./.dev.sh
```

#### Option 2 — Docker Compose directly

```
docker compose up --build
```

Wait until:

- Database health check passes.
- Backend responds on its health route.
- Frontend is listening.

**Default URLs after startup**

| Service | URL |
|---|---|
| **Frontend (dashboard)** | `http://localhost:3015` |
| **Backend API (prefix)** | `http://localhost:4515/api` |
| **Swagger (API docs)** | `http://localhost:4515/api/docs` |
| **Postgres (from host)** | host `127.0.0.1`, port **5434** (unless `DATABASE_HOST_PORT` overrides) |

### Step A4 — If something fails

- Run `docker compose ps` and confirm `laketown-db`, `laketown-backend`, `laketown-frontend` are up.
- Check logs: `docker compose logs backend` (or `frontend`, `db`).
- Confirm no other process uses ports **4515**, **3015**, or **5434**.

---

## 4. Part B — Verify the stack

### B1 — API health

```
curl -s http://localhost:4515/api/health
```

You should get a successful JSON response (exact shape depends on implementation).

### B2 — Frontend

Open a browser: `http://localhost:3015` — you should see the **login** page.

### B3 — Swagger

Open: `http://localhost:4515/api/docs` — interactive API documentation should load.

---

## 5. Part C — First login and security

### C1 — Default admin (seeded on backend bootstrap)

| Field | Value |
|---|---|
| **Username** | `admin` |

| Field | Value |
|---|---|
| **Email** | `admin@laketown.co.ke` |
| **Password** | `admin123` |

Log in on the frontend with username (or email) and password.

### C2 — Security

**Change the default password immediately** after first login (use **Settings** or your organisation’s password policy flow).

Treat `JWT_SECRET` and database passwords as **secrets** in any shared or production environment.

---

## 6. Part D — Admin setup (create everything in the right order)

**Do not skip this order.** Products must exist before recipes; branches and users should exist before daily operations.

### D1 — Create branches

1. Log in as **admin**.
2. Open **Branches** → **Add Branch**.
3. Fill **Name**, **Code**, **Type** (**General butchery** or **Pork butchery**), **Active**.
4. Save.

Repeat for each physical outlet.

### D2 — Create users and assign branches

1. Open **Users Management** → **Create User**.
2. Set **Username**, **Email**, **Password**, **Role** (e.g. **cashier** for counter staff).
3. Assign **Branch** for every **non-admin** operational user.
4. Save.

**Rule:** Cashiers must have the **correct branch** before they work.

### D3 — Create products

1. Open **Products Management** → **Add Product**.
2. Set **Name**, **Unit**, **Retail price**, **Wholesale price**, **Category**, **Sale Mode**, **Active**.

**Categories (conceptual):**

- **Raw stock** — meat, bones, bulk inputs.

- **Production item** — anything produced via recipe then sold (e.g. mutura, soup, pork fry).
- **Consumable** — operational supplies.

**Sale modes** include weight, portion, cup, piece — match how you sell at the counter.

**Rule:** If a product is the **finished output** of a recipe, create it as a **Production item**.

### D4 — Create recipes

1. Open **Recipes** → **Add Recipe**.
2. Set **Recipe Name**, **Branch Scope**, **Finished Product** (must be an existing **Production item**), **Expected Output**, **Unit**.
3. Add **ingredients** (linked products and/or free-text lines with quantities and units).
4. Save.

### D5 — Create suppliers

1. **Suppliers** → **Add Supplier** — name, contact, phone, email, address, notes, **Active**.

### D6 — Create customers

1. **Customers** → **Add Customer** — name, phone, type (**Regular**, **Credit**, **Wholesale**), limits/terms if credit, **Active**.

### D7 — Create partners (if you use stock sharing)

1. **Partners** → **Add Partner** — details and **Active**.

### D8 — Go-live checklist (before cashiers use the system)

- [ ] Branches exist and are **active**.
- [ ] Each cashier has the **right branch**.
- [ ] Raw stock products exist; **production items** exist for prepared goods.
- [ ] Recipes point to valid **finished products**.
- [ ] Suppliers exist for incoming stock.
- [ ] Customers exist (including credit/wholesale if needed).
- [ ] Partners exist if you record **stock share**.

---

## 7. Part E — Cashier day (operate one branch)

Use this sequence **every** working day.

| Step | Action |
|---|---|
| 1 | **Log in**. |

| Step | Action |
|---|---|
| 2 | **Confirm branch** in the header; if wrong, stop — admin must fix assignment. |
| 3 | Open **Stock Session** → **Open** the day’s session (do not sell before a session is open). |
| 4 | Record **opening stock** (product, quantity, unit, notes if needed). |
| 5 | Record **incoming stock** when deliveries arrive (supplier, quantities). |
| 6 | **Production** — if you make prepared items: record production runs from **Recipes** (outputs, ingredient usage, wastage). |
| 7 | **Sales** — **New Sale**; confirm branch; add lines; submit. |
| 8 | **Payments** — record payment for unpaid sales as needed. |
| 9 | **Wastage** or **stock share** (if your process uses them). |
| 10 | Record **closing stock**. |
| 11 | **Close** the stock session. |

**Permissions note: Cashiers** create operational records. **Editing or deleting** saved **sales** or **stock entries** is **admin-only** (corrections and governance).

---

## 8. Part F — Admin oversight (reports and corrections)

### F1 — Reports

Admins open **Reports** to review branch performance (sales, stock, aggregates — depends on deployment).

Use **branch** and **date** filters as available.

### F2 — Corrections

Per policy, **admins** may edit/delete **saved sales** and **stock entries** where the business allows; cashiers must escalate.

### F3 — Activity log

Admins can review **Activity Log** for audit visibility (where enabled in your build).

---

## 9. Part G — Production deployment (summary)

Full detail: see **`DEPLOYMENT.md`** in the repository root. Summary:

1. **Server:** Linux with Docker, Docker Compose, Git, SSH.

2. **DNS:** Point API and admin hostnames to your server (e.g. `apis.laketown.co.ke`, `admin.laketown.co.ke`).
3. **SSL:** Certbot or nginx-terminated TLS on the host (containers are often behind nginx).
4. **Secrets:** Create **`.env.production`** with `DATABASE_*`, `JWT_SECRET`, `JWT_EXPIRES_IN`, `FRONTEND_URL`, `NEXT_PUBLIC_API_URL` (production URLs, **`/api`** suffix on the public API URL as used by the frontend).
5. **Deploy:**

```
bash docker compose -f docker-compose.prod.yml --env-file .env.production up -d --build
```

Or use **`.deployment.sh`** per `README.md`.

6. **Verify:**

```
bash curl https://<your-api-host>/api/health
```

7. **GitHub Actions:** Configure repository secrets (`SSH_HOST`, `SSH_USER`, `SSH_PRIVATE_KEY`, `DEPLOY_PATH`, database and JWT vars, URLs) for automated deploys from the configured branch.

---

## 10. Appendix

### A. Default Docker Compose ports (local **`docker-compose.yml`**)

| Variable | Typical value |
|---|---|
| `BACKEND_HOST_PORT` | `4515` |
| `FRONTEND_HOST_PORT` | `3015` |
| `DATABASE_HOST_PORT` | `5434` |

### B. Multibranch reminder

Each **branch** is a separate operating unit. **Stock sessions** and **sales** belong to a branch. Staff are tied to a **branch**; branch **type** is **General butchery** vs **Pork butchery** for classification and reporting. See `docs/laketown/laketown-butchery.md`.

### C. Tech stack (reference)

- **Backend:** NestJS, TypeORM, PostgreSQL, JWT, Swagger.
- **Frontend:** Next.js, React, Tailwind.

### D. Troubleshooting quick list

| Symptom | Check |
| --- | --- |
| Frontend cannot reach API | `NEXT_PUBLIC_API_URL` at **build/runtime** matches backend URL; rebuild frontend if you changed API URL. |
| CORS errors | `FRONTEND_URL` in backend env matches the URL you use in the browser (including port). |
| Database connection errors | DB container healthy; credentials match `.env`; backend uses host `db` inside Compose. |
| Login fails | Backend running; seed created `admin` user; correct password after changes. |

---

*End of LakeTown Butchery End-to-End Guide.*